DUET: Dual-Granularity Address Translation for High-Capacity SSDs

Anonymous Author(s)
Affiliation omitted for double-blind review

***Abstract*—High-capacity QLC SSDs cannot keep a 4 KB page-level mapping table in on-device DRAM. Under a limited DRAM budget, existing FTLs slow reads with mapping misses, slow writes with block-level merges, or turn small writes into read-modify-writes of a uniform coarse indirection unit (IU). We present DUET, an FTL that keeps the coarse IU and maps smaller writes as 4 KB pages in a DRAM hash table. DUET spills these entries to NAND when the table fills and merges fine pages one coarse IU at a time. With 1/8 of the page-level mapping DRAM, DUET reaches 91% of the throughput of a page-level FTL with unlimited DRAM, whereas existing FTLs reach at most 71%.**

I. Introduction

The flash translation layer (FTL) is the SSD firmware that maps every logical page to its physical location. Because NAND flash cannot be overwritten without erasing a whole block, physical pages are programmed sequentially within a block and every update is written out of place, so the FTL keeps a mapping table that translates each logical page to its current physical page. Conventional SSDs map at 4 KB granularity and keep this table in on-device DRAM.

With QLC NAND, SSDs of 64 and 128 TB are being released and developed [1], [2]. At 4 KB granularity, a 64 TB SSD has 2^{34} logical pages, so each entry needs at least 34 bits and the table takes 68 GB. A 128 TB SSD needs 35-bit entries and 140 GB. Providing this much on-device DRAM is impractical because of its cost, power, and board area [3], so large SSDs have to translate addresses with a mapping DRAM budget that is only a fraction of the full page-level table (Figure 1a).

Existing techniques reduce the mapping DRAM, but each of them pays for it in latency. *On-demand mapping* [4]–[6] (Figure 1b) keeps the page-level table in flash and keeps part of it in an on-device map cache. A read that misses in the map cache must first read its mapping entry from NAND. This extra read is synchronous because the data read can start only after it completes. Read latency thus spikes whenever the map cache misses. *Block-level hybrid mapping* [7]–[9] (Figure 1c), on the other hand, maps most data per NAND block and appends updates to page-mapped log blocks. When free space runs low, the log blocks are merged back into their data blocks, and a merge must copy all valid pages of a whole NAND block. Each 3D NAND generation enlarges the NAND block [10], so a merge copies more pages, and these copies compete with host writes for NAND bandwidth. In our measurements, the p99 write latency of FAST [7], a representative hybrid mapping FTL, is 3–4× that of a page-level mapping FTL under random I/O.

Many recent commercial high-capacity SSDs instead map all data with a *uniform coarse indirection unit* (uniform coarse IU) of 16–64 KB [3], [11]. For example, Solidigm's 122.88 TB D5-P5336 maps data in 32 KB IUs [12]. Each entry maps 8× as much data as a 4 KB entry, so the mapping table needs only 1/8 of the DRAM of a page-level table. The whole coarse table fits in on-device DRAM, with no mapping misses and no block-level merges. The cost moves to small writes. When a write is smaller than a coarse IU, the FTL reads the rest of the unit, merges the new data, and writes the whole unit again

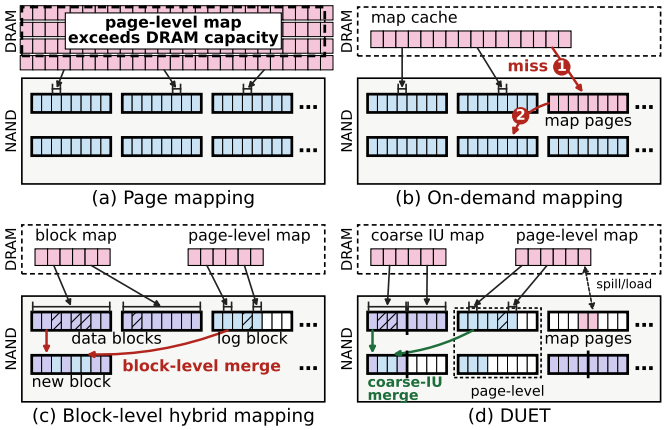


Fig. 1. Address translation of four FTLs. In (d), page-level mappings spill to map pages instead of being merged when the DRAM table is full, and are loaded back on access.

(read-modify-write, RMW). This raises the write amplification (WA) and lengthens write latency.

To keep the small table of the uniform coarse IU and still absorb small writes without RMW, our approach lets page-level mapping and coarse-IU mapping coexist. This raises two challenges. First, the coarse map already takes much of the DRAM budget, so little DRAM is left for page-level mappings. The FTL needs a way to handle the page-level mappings that outgrow it without merging them into coarse IUs too often, which raises WA. Second, with two mapping granularities, the FTL must decide where to place each kind of data. In garbage collection (GC), it must also decide which kind of block to reclaim and whether to merge page-mapped data back into coarse IUs. In our experiments, these decisions change WA by up to 1.7×.

We present **DUET** (**DU**al-granularity **E**conomical **T**ranslation), an FTL that keeps two mapping granularities, the coarse IU and the 4 KB page, under a fixed mapping DRAM budget (Figure 1d). DUET maps most data per coarse IU and writes small writes out of place as 4 KB pages, so the host write path has no RMW. All coarse-IU mappings and most page-level mappings stay in on-device DRAM, so DUET needs far fewer mapping reads than on-demand mapping. When the page-level mappings outgrow the DRAM budget, DUET stages them in NAND instead of merging them right away. DUET writes small and large writes to separate blocks, and its GC decides which kind of block to reclaim. It also decides by cost-benefit whether to copy the 4 KB pages or merge them into their coarse IU. A merge rewrites only one coarse IU, so DUET never performs block-level merges, and the cost of a merge stays the same as the NAND block grows.

We implement DUET and six other FTLs in NVMeVirt [13], a kernel-level NVMe SSD emulator, and evaluate them under the mapping DRAM of a 32 KB uniform coarse IU with synthetic raw I/O, real-world traces, and applications. On average, DUET reaches 91% of the throughput of a page-level FTL with unlimited DRAM, whereas on-demand mapping, block-level hybrid mapping, and a

uniform coarse IU reach at most 71%, 56%, and 70%, respectively. Compared with on-demand mapping, DUET cuts the mapping reads per host read from 0.26–0.47 to 0.02, so its average read latency is 39–57% lower. Because it merges one coarse IU at a time and writes small writes without RMW, its WA is 53–65% lower than that of block-level hybrid mapping and 41% lower than that of the uniform coarse IU.

II. Background and Motivation

A. Address Translation under a DRAM Budget

NAND flash is programmed in pages but erased only in much larger blocks, so the FTL writes every update out of place and maps each logical page to its current physical page. Garbage collection (GC) reclaims a block by copying its valid pages elsewhere and erasing it. These copies make NAND writes exceed host writes, and the ratio of NAND writes to host writes is the write amplification (WA). The over-provisioned (OP) space, which the host cannot see, gives GC spare room.

A page-level FTL keeps one entry per 4 KB logical page, so its table is about 0.1% of the capacity. Providing this much on-device DRAM in a high-capacity SSD [1], [2] is impractical because of its cost, power, and board area [3], so the FTL of a high-capacity SSD must either keep part of the mapping in flash or map data in larger units. Each of the approaches below suffers from an overhead in a different part of the I/O path.

On-demand mapping. On-demand mapping stores the page-level table in non-volatile NAND flash as flash translation pages (FTPs) and keeps only recently used entries in an on-device *map cache*, as introduced by DFTL [4]. A small global translation directory (GTD) locates each FTP. A read that misses in the map cache must first read the FTP and only then the data page, so two flash reads are serialized on the read path. Evicting a dirty entry also writes its FTP back, which adds write amplification. LeaFTL [5] sorts buffered writes by logical page number so that consecutive logical pages are placed on consecutive physical pages, and compresses the map cache into learned linear *segments*, each of which predicts the locations of such a run. The same DRAM size thus covers more pages. Under small random writes, however, each segment covers only a few pages, and a mispredicted location imposes an extra flash read. μ-FTL [14] maps data at multiple granularities with variable-size extents, but it keeps the extents in a B+-tree in flash and caches only part of the tree in DRAM. For randomly written data, these FTLs miss often in the map cache, so their read latency can spike.

Block-level hybrid mapping. Block-level hybrid FTLs such as FAST (fully-associative sector translation) [7] and Janus-FTL [8] map most data per NAND block, which needs only one entry per block, and absorb updates in a small pool of page-mapped *log blocks*. When the pool runs out, a log block is reclaimed by *merging*. The valid pages of a logical block are copied from its data block and the log block into a free block, and the old data block and log block are erased. A merge imposes a higher overhead than the GC of a page-level FTL. Page-level GC picks the block with the fewest valid pages, so it copies few pages per reclaimed block. A merge instead rewrites the whole data block even when only a few of its pages were updated. One log block can also hold updates to many logical blocks [7]. To reclaim such a log block, the FTL merges every logical block that has an update in it, and each merge copies the valid pages of a whole logical block to a new block.

The merge copies compete with host writes for NAND bandwidth, which raises write latency and lowers write throughput. The size of a NAND block keeps growing with 3D NAND generations [10]. A larger log block holds updates to more logical blocks, so reclaiming it involves more merges, and each merge copies more pages. CoMap [15] selects page or block mapping for each write request with reinforcement learning and caches the page-level mappings on demand, so it can suffer both mapping misses and block-level merges.

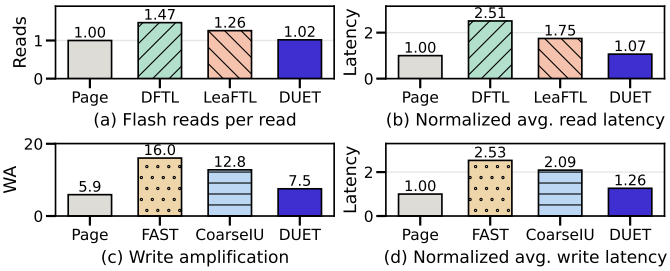


Fig. 2. Existing FTLs under the mapping DRAM of a 32 KB uniform coarse IU, geometric mean over 9 workloads. Latency is normalized to Page. (a) Flash reads per host read. (b) Average read latency. (c) Write amplification. (d) Average write latency.

Uniform coarse-IU mapping. High-capacity SSDs in production simply map all data with a *uniform coarse indirection unit* (uniform coarse IU) [3], [11]. For example, Solidigm's 122.88 TB D5-P5336 uses a 32 KB IU [12]. Since one entry maps 32 KB of contiguous space, the table needs about 1/8 of the DRAM of a 4 KB page-level table and fits in DRAM. Every read thus takes a single flash access, and there are no block-level merges.

The cost moves to writes smaller than a coarse IU. When a write smaller than a coarse IU arrives, the FTL must read the rest of the unit, merge the new data, and program the whole unit again (read-modify-write, RMW). The extra reads lengthen the write path, and the rewritten pages raise write amplification by up to the coarse-IU size over the write size. A write buffer in the SSD merges small writes to the same coarse IU. If these writes do not fill the coarse IU before they are evicted from the buffer, however, the FTL still reads the missing pages and rewrites the whole unit.

B. Motivation

Figure 2 compares the three approaches that reduce the on-device DRAM with a page-level FTL that keeps its whole table in DRAM. Every FTL except the page-level FTL (*Page*) gets the mapping DRAM of a uniform 32 KB coarse IU, which is 1/8 of that of Page. Each bar is the geometric mean over the 9 workloads of Section V (synthetic raw I/O, real-world traces, and applications).

On-demand mapping slows reads. With mapping misses, DFTL needs 1.47 flash reads per host read and LeaFTL 1.26 (Figure 2a). Their average read latency is 2.5× and 1.7× that of the page-level FTL (Figure 2b) because each miss adds a synchronous flash read before the data read.

Block-level hybrid mapping and uniform coarse IUs slow writes. FAST's block-level merges raise WA by 2.7× (Figure 2c), so its average write latency is 2.5× that of the page-level FTL (Figure 2d). The uniform coarse IU avoids merges, but every 4 KB write becomes an RMW of its 32 KB unit, so its WA grows by 2.2× and its write latency by 2.1×.

DUET uses the same DRAM budget. It lowers read latency by issuing fewer flash reads per host read, and it lowers write latency by merging efficiently, which lowers WA.

Goal. An FTL that fits a limited DRAM budget should (i) minimize mapping reads, so that a host read almost always costs one flash read, (ii) eliminate block-level merges, and (iii) absorb small writes without RMW. DUET meets all three by keeping the coarse IU and adding a fine-grained level in the same budget.

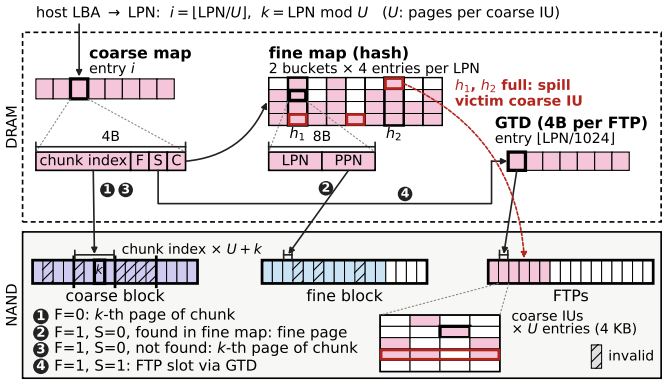


Fig. 3. DUET mapping structures and read path.

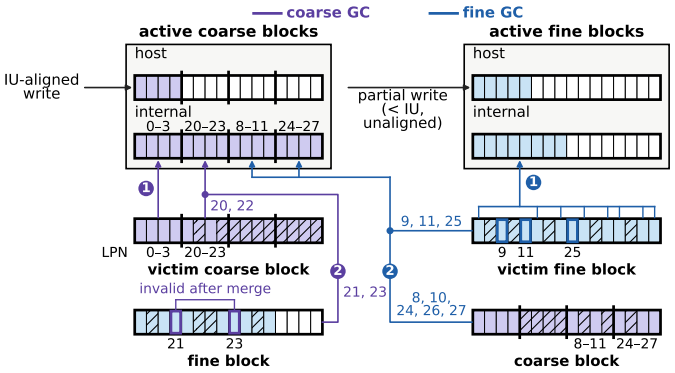


Fig. 4. DUET write and reclamation paths. Coarse GC (purple) and fine GC (blue) either (1) copy valid data or (2) merge it into new chunks.

III. DUET Design

DUET maps every logical page at two mapping granularities (Figure 3). A *coarse map* keeps one entry per coarse IU, and a *fine map* maps the 4 KB fine pages written for parts of a coarse IU. The coarse map is always in DRAM and maps every page except those currently stored as fine pages, so most reads need no flash read to find their mapping. Part of the fine map can move to NAND, but unlike demand-paging FTLs [4], [5], [14], most of the map stays in DRAM, so far fewer reads need a second flash read. Unlike hybrid FTLs [7], which merge one block at a time, DUET merges fine pages one coarse IU at a time, so each merge costs less. Unlike coarse-IU FTLs [3], [11], DUET writes small writes as fine pages without RMW.

A. Two Mapping Granularities

Coarse map. The logical space is divided into coarse IUs of U pages ($4U$ KB), so a logical page number (LPN) belongs to coarse IU $i = \lfloor \mathrm{LPN}/U \rfloor$ at page offset $k = \mathrm{LPN} \bmod U$. Each coarse IU is stored as a *chunk* of U physically consecutive, U-page-aligned pages in a *coarse block*, and its 4 B coarse entry holds the chunk index. The coarse map takes $1/U$ of the DRAM of a page-level table, because an entry of the same size covers U times as many pages.

Fine map. A write smaller than a coarse IU is written out of place as 4 KB *fine pages* into separate *fine blocks*. The fine map is a two-choice bucketized hash table [16]. Each 8 B entry holds an LPN and the physical page number (PPN) of its fine page. Each LPN has two candidate buckets of four entries each, and a new entry goes to the less loaded bucket. Reading a 32 B bucket costs one random DRAM access [17], [18]. Placing each entry in the less loaded of two candidate buckets spreads the entries far more evenly than a single candidate bucket [16]. A bucket thus overflows much less often, so DUET evicts entries less often. The table takes all of the DRAM budget that the coarse map leaves.

The table is similar to cuckoo hashing [19] in that each entry has two candidate buckets. Unlike cuckoo hashing, DUET never moves an existing entry to its other bucket to make room, so an insert does not trigger a chain of moves. When both candidate buckets are full, DUET instead writes the fine-map entries of a victim coarse IU to a *fine translation page* (FTP) in NAND and frees them from the table (*spill*, Section III-D). A table lookup therefore checks at most the eight entries of the two candidate buckets, and an insert writes into a free entry of one of them. A spilled entry has a fixed location in NAND, so one map-page read returns it.

Entry size for large-capacity SSDs. Even at 128 TB, an entry in the fine map fits in 8 B. As in quotienting [20], its bucket index already encodes most of the LPN, so the entry stores only the remaining LPN bits, a candidate-bucket bit, and the PPN. A coarse entry grows to 5 B to hold a 32-bit chunk index and its flag bits, but a page-level table then also needs 5 B entries, so the coarse map still takes $1/U$ of its DRAM.

Flag bits. Each coarse entry also has three flag bits. The *has-fine-pages* (F) bit tells whether the coarse IU has fine pages, the *spilled* (S) bit tells whether their fine-map entries are in NAND, and the *clean* (C) bit tells whether the entries in the table are identical to their copy in NAND (Section III-D).

B. Host I/O Path

Read path. As shown in Figure 3, a read first checks the coarse entry of coarse IU i, and its flag bits select one of four cases. If F is clear, DUET reads page k of the chunk (❶). If F is set and S is clear, DUET looks up the LPN in the fine map. It reads the fine page when the LPN is found (❷), and page k of the chunk otherwise (❸). If S is set, the global translation directory (GTD) [4] locates the FTP of the LPN, and DUET reads its slot (❹), then the fine page that the slot points to, or page k of the chunk when the slot is empty. Unless the coarse IU is spilled, a read thus reads at most three DRAM locations, the coarse entry and the two candidate buckets, each with one random DRAM access. Only a read of a spilled coarse IU adds one map-page read from NAND.

Write path. Host writes go to two *active blocks*, an active coarse block for chunks and an active fine block for fine pages. A host write is split at coarse-IU boundaries. Each *IU-aligned write* is written as a new chunk in the active coarse block. The old chunk and all fine pages of the coarse IU become invalid, and the corresponding fine-map entries are removed. If the coarse IU already has fine pages, DUET instead writes the IU-aligned write as fine pages, because part of the coarse IU was written relatively recently and the coarse IU is likely to be split again soon.

A *partial write*, which is smaller than a coarse IU, is written as fine pages into the active fine block. DUET inserts the LPN and PPN of each fine page into the fine map. If the LPN already has a fine page, DUET updates the PPN of its fine-map entry instead, and the old fine page becomes invalid.

DUET keeps fine pages and chunks in different blocks. Fine pages hold small and hot data that was written recently, and chunks hold data that was written long ago. Mixing data of different update frequencies in one NAND block raises the GC cost [21]–[23]. As in other FTLs [24], DUET also separates host writes from internal writes (GC and merges), so it keeps four active blocks (Figure 4). When an active block is full, DUET takes a free block as the new active block.

C. Reclaiming Path

DUET reclaims a block when the free blocks run low, and it has two kinds of GC. *Fine GC* reclaims a fine block, and *coarse GC*

reclaims a coarse block. To find the valid data of a victim, DUET keeps a valid bitmap with one bit per chunk slot of the coarse blocks and one bit per page of the fine blocks.

Fine-block cap. A coarse IU with fine pages still keeps its whole chunk in a coarse block, even though some pages of the chunk are no longer valid. In the worst case, no coarse IU is entirely in fine pages, and every coarse IU has part of its data in its chunk and the rest in fine blocks. The coarse blocks must therefore be able to hold the whole logical capacity. Considering this case, DUET sets the *fine-block cap* to the OP space minus the blocks reserved for GC, the active blocks, and the blocks that hold FTPs. Below this cap, the fine blocks grow as partial writes add fine pages and shrink when the victim selection below reclaims fine blocks.

Coarse-or-fine victim selection. When the fine blocks reach the fine-block cap, DUET reclaims a fine block. Otherwise, it picks whichever of the coarse and fine victim blocks has fewer valid pages to copy.

Coarse GC. Coarse GC handles each valid chunk of the victim in one of two ways (purple lines in Figure 4). A chunk whose coarse IU has no fine pages (LPN 0–3) is copied unchanged to a new chunk (1). A chunk whose coarse IU has fine pages (LPN 20–23) is only partly valid, so DUET merges it (2). DUET looks up the LPNs of the coarse IU in the fine map. An LPN found there has a fine page (LPN 21 and 23), and an LPN not found there is still valid in the chunk (LPN 20 and 22). DUET reads these pages and writes them as one merged chunk. The fine pages then become invalid, their fine-map entries are removed, and the F bit of the coarse IU is cleared.

Fine GC. Fine GC handles each coarse IU with valid fine pages in the victim in one of two ways (blue lines in Figure 4). DUET either copies the fine pages to another fine block (1) or merges the coarse IU with its chunk (2). In Figure 4, LPN 9 and 11 are merged with the chunk of LPN 8–11, and LPN 25 with the chunk of LPN 24–27. Let d be the number of fine pages of the coarse IU in all fine blocks, not only in the victim. A merge reads the chunk and these d fine pages and writes one new chunk. All d fine pages then become invalid, and their fine-map entries are removed.

Copy-or-merge decision. DUET chooses one of the two ways by a cost-benefit model. The cost is the number of pages written, and the benefit is the number of pages freed in the fine blocks. Let P be the number of pages per block, v the number of valid pages in the victim, and U the number of pages in a coarse IU. For the same victim, copying costs v page writes and frees $P - v$ pages. A merge of a coarse IU with d fine pages instead costs U page writes, one coarse IU, and frees its d fine pages. The cost-benefit value, that is, the number of pages freed in the fine blocks per page written, is thus $(P-v)/v$ for copying (1) and d/U for a merge (2). DUET copies the fine pages when $(P - v)/v \geq d/U$, and otherwise merges the coarse IU. DUET makes this decision for every coarse IU that has valid fine pages in the victim fine block. Copying keeps the fine-map entries and a merge frees them, so the fine map grows or shrinks with the workload (Section V).

D. Fine-Map Spill and Load

Under a fixed DRAM budget, the table cannot index every fine page that the fine blocks can hold. The DRAM capacity is fixed in hardware, so a firmware update cannot enlarge it. For example, a commercial 122.88 TB SSD [12] has DRAM for 1/8 of a page-level table. When DUET runs within this budget, the coarse map takes about half of it, so only about 6% of the DRAM of a page-level table is left for the fine map. In our evaluation setup, this indexes about 72% of the fine pages that the fine blocks can hold at the fine-block cap.

Another challenge is hash collisions. An insert can find both candidate buckets full while other buckets still have free slots. Hashing schemes that place a colliding entry in another bucket [19], [25]–[27] make a lookup probe more buckets or an insert move existing entries. This takes more DRAM accesses. Merging the coarse IU of a colliding entry causes extra NAND writes.

DUET instead keeps the fine-map entries of some coarse IUs in FTPs in NAND, so that DUET can map more fine pages in the fine blocks without more DRAM. The fine-map entries of a coarse IU move to its FTP when the table fills (*spill*) and back to the table when the coarse IU is accessed again (*load*), and no data moves in either direction. Only fine pages have fine-map entries, so the spilled entries map at most the fine pages at the fine-block cap, which are 4.3% of the logical capacity in our evaluation setup.

Spill. As shown in Figure 3, the fine-map entry of each LPN has a fixed 4 B slot in an FTP, which holds the entries of 1024 consecutive LPNs. As in DFTL, the GTD locates each FTP with one 4 B entry. When an insert finds both candidate buckets full, DUET picks a victim among the coarse IUs that have an entry in the two buckets. It writes all entries of the victim to its FTP, sets its S bit, and frees its table entries. The other coarse IUs of the same FTP are spilled with the same program.

Load. An access to a spilled coarse IU reads its FTP and loads only the entries of that coarse IU into the table. The other coarse IUs of the FTP stay spilled. A write loads the entries and then updates them. A read reads the FTP before the data, loads the entries, and sets the C bit of the coarse IU. The coarse IU stays clean until one of its entries changes.

What to spill. DUET picks a clean coarse IU first to reduce FTP writes to NAND. A clean coarse IU is simply dropped from the table and marked spilled because its FTP copy is still valid. If there is no clean coarse IU, DUET picks the coarse IU with the most fine pages, so one FTP program frees the most table slots.

GC. When GC moves fine pages of spilled coarse IUs, it reads and programs each FTP at most once per victim block to record the new locations of these fine pages. A merge of a spilled coarse IU reads its entries from the FTP and then clears its F and S bits.

E. Crash Recovery

High-capacity SSDs are mainly deployed as enterprise and data center SSDs, whose power-loss capacitors write buffered data and logs to NAND on a power failure [12], [28]. DUET follows the crash recovery of conventional SSDs, which log the LPN of every written page in NAND and rebuild the mappings by scanning these logs in write order [29]. At power-on, the LPNs of a coarse IU that fill a whole U-page chunk are recovered into the coarse map, and the other LPNs into the fine map. Each FTP also records its index in its out-of-band (OOB) area, so the same scan rebuilds the GTD.

IV. Implementation

We implement DUET and six other FTLs in NVMeVirt [13], a kernel-level NVMe SSD emulator that models the latency of every NAND read, program, and erase over its channels and dies. We add a common FTL interface to NVMeVirt. Each FTL implements the host read, write, and flush handlers and its own GC, and it issues every data and mapping access as a flash operation. Mapping reads and programs therefore take NAND time like data accesses and are counted in WA. All FTLs share the same NAND model, write buffer, and host interface and differ only in the FTL. Based on the NVMeVirt code, we also let host reads suspend programs and erases and pace

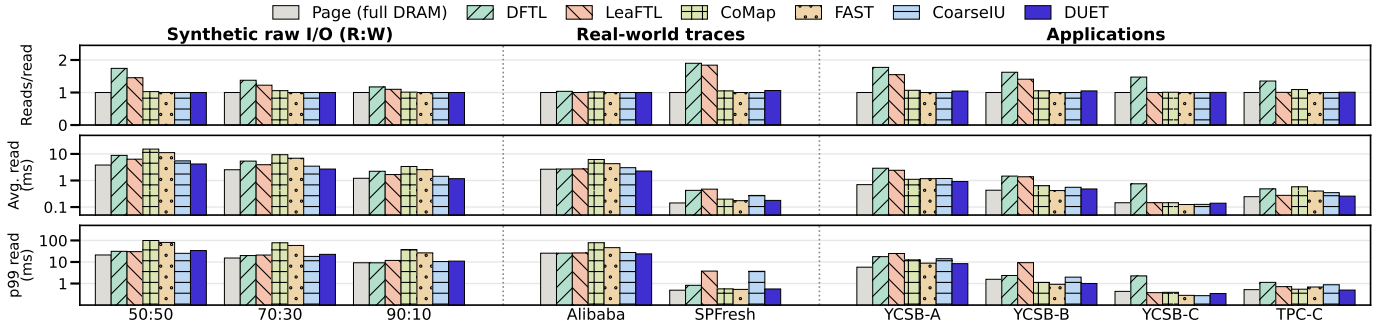


Fig. 5. Read path. From top, flash reads per host read (data plus mapping reads), and average and p99 read latency.

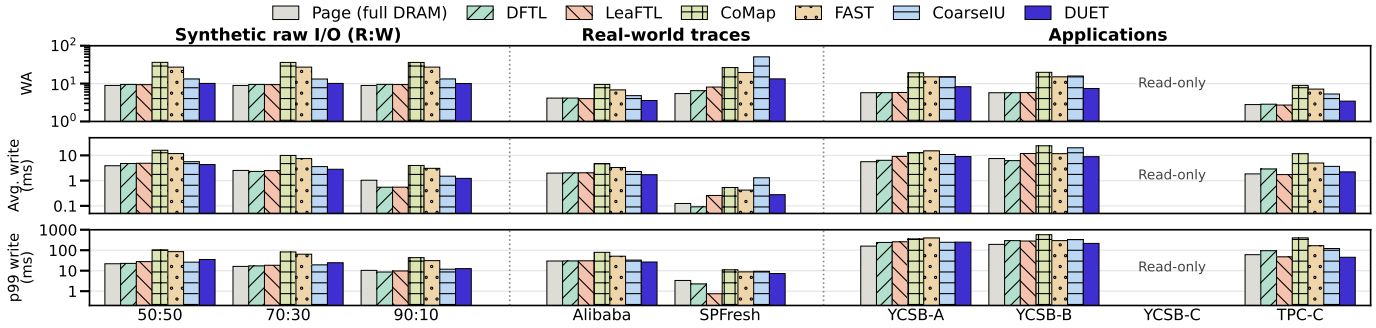


Fig. 6. Write path. From top, device write amplification, and average and p99 write latency. YCSB-C is read-only.

background NAND operations to mimic real SSDs. These changes apply equally to all FTLs. DUET takes about 1,600 lines of C.

V. Evaluation

A. Experimental Setup

Setup. We run all experiments on a server with an Intel Xeon Gold 6342 CPU and Linux kernel 6.8. The emulated SSD has eight channels with two dies each and exposes 32 GB to the host. Its OP space is 7% of that capacity. All workloads and FTLs use this device. We cap the mapping DRAM of every FTL except the page-level mapping FTL at the DRAM of a uniform 32 KB coarse IU, an IU size used in production high-capacity SSDs [3], [11], which is 4 MB. The page-level mapping FTL uses 32 MB. DUET uses 64 KB coarse IUs. Its budget holds a 2.0 MB coarse map, a 2.0 MB fine-map table with 261,256 entries, a 32 KB GTD, and 0.1 MB of valid bitmaps for the chunks and the fine blocks.

Workloads. We run three types of workloads, synthetic raw I/O, real-world traces, and applications. The synthetic raw I/O issues requests to the block device directly, without a file system. The requests are uniformly random over the whole device. Of the requests, 70% are 4 KB and 30% are 128 KB, each aligned to its size, and the read ratio is 50%, 70%, or 90%. The real-world traces are the Alibaba cloud block traces [30] and an I/O trace of SPFresh [31], a vector search index with in-place updates. From the Alibaba traces, we take the volumes whose drive writes per day are 0.1 to 1, which write 8.2 TB in total. To fit them in the 32 GB device, we keep 1.3% of their address space by spatial sampling [32], together with every request to the sampled space. We replay the SPFresh trace without sampling until it writes 64 GB, twice the device capacity, which also reads 1.2 TB. The raw I/O and the traces run with 32 outstanding requests, and we measure the second half of each run. The applications are YCSB workloads A, B, and C [33] and TPC-C [34] on PostgreSQL and ext4, each with 64 client threads. They run in a memory cgroup of 6.4 GB, 20% of the device, which limits the page cache.

Baselines. We compare DUET with six other FTLs, which cover the three classes of Section II. **Page** maps every 4 KB page and keeps the whole table in DRAM, without a DRAM limit. **DFTL** [4] caches mapping entries in an LRU map cache and writes back the dirty entries of a translation page together. Its budget holds a 32 KB GTD, a 1.1 MB valid-page bitmap for GC, and a 3.0 MB map cache. **LeaFTL** [5] learns linear segments, each of which predicts the physical pages of consecutive LPNs, caches them in DRAM, and sorts writes by LPN to form longer segments. Its budget holds a 128 KB GTD, a 1.1 MB valid-page bitmap, and 2.9 MB of segments. **FAST** [7] maps data blocks per NAND block and appends updates to one sequential and several random log blocks, and we limit the number of log blocks so that their page-level maps fit the DRAM budget, which allows 16 log blocks. **CoMap** [15] chooses page-level or block-level mapping for each write request with a Q-learning agent. Its budget holds a 1 MB bitmap of page-mapped pages, a 32 KB GTD, 2.9 MB of page-level maps for eight log blocks, and a 0.2 MB map cache. **CoarseIU** maps all data with a uniform 32 KB coarse IU and keeps the whole 4.0 MB table and a 0.1 MB valid bitmap in DRAM. Small writes to the same coarse IU are merged in the 1 MB write buffer, and a partial coarse IU is written with RMW only when it leaves the buffer. Its DRAM sets the budget that every other bounded FTL receives.

B. Read Latency

Figure 5 shows the read path. DUET reads the mapping from NAND only for spilled coarse IUs, so it needs 1.02 flash reads per host read on average and at most 1.06, on SPFresh. DFTL and LeaFTL miss in their map caches and need up to 1.90 and 1.84 flash reads on SPFresh. As a result, the average read latency of DUET is 1.07× that of Page on average, whereas that of DFTL and LeaFTL is 2.51× and 1.75× that of Page. The average read latency of DUET is 34%, 49%, and 20% lower than that of FAST, CoMap, and CoarseIU on average. DUET writes small writes to fine blocks without RMW and merges one coarse IU at a time, so its reads wait less behind writes. The p99 read latency shows a similar trend, and DUET has the lowest among the FTLs with a limited DRAM budget.

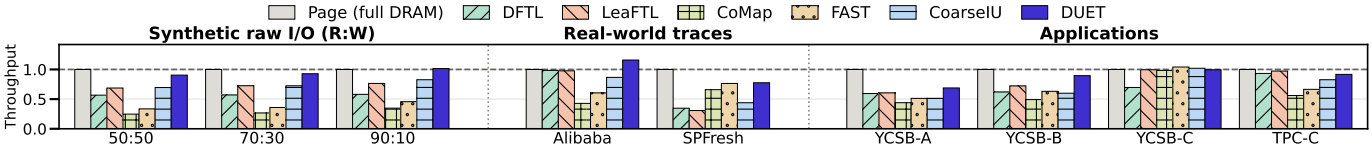


Fig. 7. Throughput normalized to Page.

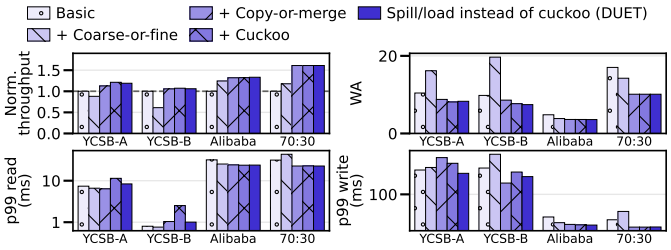


Fig. 8. Ablation. The two GC decisions are added to Basic one by one, and the two rightmost bars handle a full fine map by cuckoo hashing or by spill and load (DUET). Throughput is normalized to Basic.

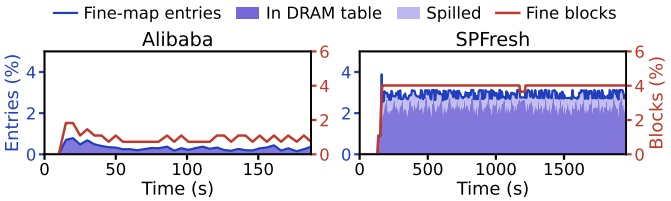


Fig. 9. Fine-map entries, as a percentage of all logical pages, and fine blocks, as a percentage of all blocks, of DUET over time, sampled every 5 s.

C. Write Amplification and Write Latency

Figure 6 shows the write path. YCSB-C is read-only, so it is left out. DUET writes small writes out of place as fine pages and merges them one coarse IU at a time, so its WA stays close to that of Page, with a geometric mean of 1.28× Page. FAST and CoMap merge whole blocks, and CoarseIU rewrites a 32 KB unit for every smaller write. Their WA is 2.7×, 3.6×, and 2.2× that of Page on average. As a result, the average write latency of CoarseIU, FAST, and CoMap is 2.1–3.6× that of Page, whereas that of DUET is 1.26×. The p99 write latency shows a similar trend. DFTL and LeaFTL also write out of place at 4 KB granularity and keep their write latency close to that of Page, but they pay for it on the read path.

On Alibaba, the WA of DUET is 14% lower than that of Page. In this workload, small writes tend to be hot. DUET writes the writes smaller than a coarse IU to fine blocks, apart from coarse blocks, so it separates hot data from cold data as data placement schemes do [24], [35], [36]. Page writes all data to one active block, so hot and cold pages mix in a block and its GC copies more valid pages.

D. Throughput

Figure 7 shows the throughput normalized to Page. DUET reaches 91% of the throughput of Page on average, whereas LeaFTL, CoarseIU, DFTL, FAST, and CoMap reach 71%, 70%, 63%, 56%, and 45%. On Alibaba, DUET is 16% faster than Page because its WA is 14% lower. DUET is the slowest relative to Page on YCSB-A and SPFresh, at 69% and 78%, where its WA is 1.4× and 2.4× that of Page and reads of spilled coarse IUs add mapping reads.

E. Ablation Study

Figure 8 adds the techniques of DUET to Basic on YCSB-A, YCSB-B, Alibaba, and 70:30. Basic fills the fine blocks up to the fine-block cap, and its fine GC always merges the fine pages of a victim into their coarse IUs. Coarse-or-fine victim selection shrinks the fine area, which helps Alibaba and 70:30 but raises WA on YCSB-A and YCSB-B. There, a merge copies the fine pages with the rest of their coarse IU, and new fine pages soon invalidate part of it. Copy-or-merge improves this. Together, the two decisions lower WA by up to 1.7× and raise the throughput by up to 1.61× over Basic.

Cuckoo makes room in full candidate buckets by moving entries along a path to a free entry [19], [37] and merges only when no path exists. It thus merges less often than copy-or-merge and lowers WA by 8% and 11% on YCSB-A and YCSB-B, where the fine map fills. Its search for a free entry on writes, however, probes 179 and 203 buckets on average, which raises the p99 read latency by 1.8× and 2.4× and the p99 write latency on YCSB-B by 1.6×. DUET instead spills entries to NAND without searching and keeps the lower WA. Compared with *Cuckoo*, DUET lowers the p99 read latency by 27% and 60% and the p99 write latency by 35% and 18% on YCSB-A and YCSB-B.

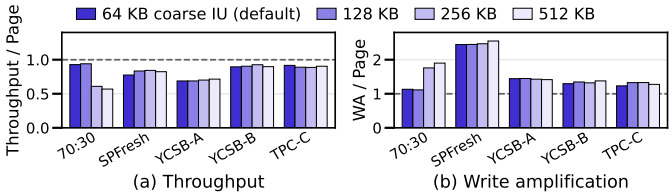


Fig. 10. DUET with coarse IUs of 64–512 KB under the same DRAM budget, normalized to Page.

F. Fine-Area Size over Time

Figure 9 shows how the fine area of DUET changes over time. Fine-map entries are shown as a percentage of all logical pages. The fine area grows and shrinks differently in each workload. On SPFresh, all writes are smaller than a coarse IU, so the fine blocks stay at their limit, 4.0% of all blocks, and fine-map entries stay at 2.7–3.1% of the logical pages. On Alibaba, 79% of the written data comes in writes of 64 KB or larger, so the fine area stays smaller. It peaks early in the run at 1.8% of all blocks and 0.8% of the logical pages, and then shrinks to 0.7–1.1% of all blocks and 0.2–0.4% of the logical pages. In both workloads, spilled entries stay within 1.4% of the logical pages, and Alibaba spills none.

G. Coarse-IU Size

Figure 10 varies the coarse IU of DUET from 64 KB to 512 KB under the same DRAM budget. A larger coarse IU shrinks the coarse map and leaves more DRAM to the fine map, so fewer fine-map entries spill. On SPFresh, the flash reads per host read drop from 1.06 to 1.00, and the throughput rises from 78% to up to 84% of Page. A larger coarse IU, however, turns more writes into fine pages. On 70:30, the 128 KB writes still cover a 128 KB coarse IU, but become fine pages from 256 KB on. Its WA rises from 1.1× to 1.9× that of Page, and its throughput drops from 93% to 57%. On YCSB-A, YCSB-B, and TPC-C, the throughput changes by less than 4% across the four sizes. The coarse IU thus works best when it is no larger than the large writes of the workload.

VI. Conclusion

DUET maps most data with coarse IUs and small writes with 4 KB fine pages, spills fine-map entries to NAND when its DRAM table fills, and merges fine pages one coarse IU at a time. With about 1/8 of the page-level mapping DRAM, DUET reaches 91% of the throughput of a page-level FTL with unlimited DRAM.

References

[1] Solidigm, "Solidigm extends AI portfolio leadership with the introduction of 122TB drive, the world's highest capacity PCIe SSD," Press release, Nov. 2024.

[2] AnandTech, "Samsung's 128 TB-class BM1743 enterprise SSD displayed at FMS 2024," https://www.anandtech.com/show/21526, Aug. 2024.

[3] L. Bert, "Real life workloads allow more efficient data granularity and enable very large SSD capacities," Micron Technology blog, Sep. 2023.

[4] A. Gupta, Y. Kim, and B. Urgaonkar, "DFTL: A flash translation layer employing demand-based selective caching of page-level address mappings," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2009.

[5] J. Sun, S. Li, Y. Sun, C. Sun, D. Vucinic, and J. Huang, "LeaFTL: A learning-based flash translation layer for solid-state drives," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[6] D. Park, B. Debnath, and D. H. C. Du, "CFTL: A convertible flash translation layer adaptive to data access patterns," in *Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, 2010.

[7] S.-W. Lee, D.-J. Park, T.-S. Chung, D.-H. Lee, S. Park, and H.-J. Song, "A log buffer-based flash translation layer using fully-associative sector translation," *ACM Transactions on Embedded Computing Systems*, 2007.

[8] H. Kwon, E. Kim, J. Choi, D. Lee, and S. H. Noh, "Janus-FTL: Finding the optimal point on the spectrum between page and block mapping schemes," in *Proceedings of the ACM International Conference on Embedded Software (EMSOFT)*, 2010.

[9] D. Jung, J.-U. Kang, H. Jo, J.-S. Kim, and J. Lee, "Superblock FTL: A superblock-based flash translation layer with a hybrid address translation scheme," *ACM Transactions on Embedded Computing Systems*, vol. 9, no. 4, pp. 40:1–40:41, 2010.

[10] C.-Y. Liu, J. B. Kotra, M. Jung, and M. T. Kandemir, "PEN: Design and evaluation of partial-erase for 3D NAND-based high density SSDs," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2018.

[11] Solidigm and Wiwynn, "Platform optimization for performance and endurance," Solidigm solution brief, Aug. 2025.

[12] Solidigm, "Solidigm D5-P5336 product specification," Rev. 006, Dec. 2024.

[13] S.-H. Kim, J. Shim, E. Lee, S. Jeong, I. Kang, and J.-S. Kim, "NVMeVirt: A versatile software-defined virtual NVMe device," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2023.

[14] Y.-G. Lee, D. Jung, D. Kang, and J.-S. Kim, "μ-FTL: A memory-efficient flash translation layer supporting multiple mapping granularities," in *Proceedings of the ACM International Conference on Embedded Software (EMSOFT)*, 2008.

[15] Y. Zhou, F. Wang, and D. Feng, "How to manage the mapping table of large-capacity solid-state drives," in *Proceedings of the Design, Automation and Test in Europe Conference (DATE)*, 2026.

[16] Y. Azar, A. Z. Broder, A. R. Karlin, and E. Upfal, "Balanced allocations," *SIAM Journal on Computing*, vol. 29, no. 1, pp. 180–200, 1999.

[17] Samsung Electronics, "8Gb C-die DDR4 SDRAM x16 (K4A8G165WC) datasheet," Rev. 1.5, Apr. 2017.

[18] S. Rixner, W. J. Dally, U. J. Kapasi, P. Mattson, and J. D. Owens, "Memory access scheduling," in *Proceedings of the 27th International Symposium on Computer Architecture (ISCA)*, 2000, pp. 128–138.

[19] R. Pagh and F. F. Rodler, "Cuckoo hashing," *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.

[20] M. A. Bender, M. Farach-Colton, R. Johnson, R. Kraner, B. C. Kuszmaul, D. Medjedovic, P. Montes, P. Shetty, R. P. Spillane, and E. Zadok, "Don't thrash: How to cache your hash on flash," *Proceedings of the VLDB Endowment*, vol. 5, no. 11, pp. 1627–1637, 2012.

[21] B. Van Houdt, "On the necessity of hot and cold data identification to reduce the write amplification in flash-based SSDs," *Performance Evaluation*, vol. 82, pp. 1–14, 2014.

[22] B. Van Houdt, "Performance of garbage collection algorithms for flash-based solid state drives with hot/cold data," *Performance Evaluation*, vol. 70, no. 10, pp. 692–703, 2013.

[23] J.-U. Kang, J. Hyun, H. Maeng, and S. Cho, "The multi-streamed solid-state drive," in *Proceedings of the USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2014.

[24] Q. Wang, J. Li, P. P. C. Lee, T. Ouyang, C. Shi, and L. Huang, "Separating data via block invalidation time inference for write amplification reduction in log-structured storage," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2022.

[25] A. Kirsch and M. Mitzenmacher, "The power of one move: Hashing schemes for hardware," *IEEE/ACM Transactions on Networking*, vol. 18, no. 6, pp. 1752–1765, 2010.

[26] M. Herlihy, N. Shavit, and M. Tzafrir, "Hopscotch hashing," in *Proceedings of the International Symposium on Distributed Computing (DISC)*, ser. LNCS, vol. 5218, 2008, pp. 350–364.

[27] P. Celis, P.-Å. Larson, and J. I. Munro, "Robin hood hashing," in *Proceedings of the IEEE Symposium on Foundations of Computer Science (FOCS)*, 1985, pp. 281–288.

[28] Open Compute Project, "Datacenter NVMe SSD specification," Version 2.7, Jan. 2026.

[29] Y. Zhou, Q. Wu, F. Wu, H. Jiang, J. Zhou, and C. Xie, "Remap-SSD: Safely and efficiently exploiting SSD address remapping to eliminate duplicate writes," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2021, pp. 187–202.

[30] J. Li, Q. Wang, P. P. C. Lee, and C. Shi, "An in-depth analysis of cloud block storage workloads in large-scale production," in *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*, 2020.

[31] Y. Xu *et al.*, "SPFresh: Incremental in-place update for billion-scale vector search," in *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[32] C. A. Waldspurger, N. Park, A. Garthwaite, and I. Ahmad, "Efficient MRC construction with SHARDS," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2015.

[33] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2010.

[34] Transaction Processing Performance Council, "TPC benchmark C standard specification, revision 5.11," 2010.

[35] S. Oh, J. Kim, S. Han, J. Kim, S. Lee, and S. H. Noh, "MiDAS: Minimizing write amplification in log-structured systems through adaptive group number and size configuration," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2024, pp. 259–275.

[36] J. Kim, S. Oh, J. Kim, J. Park, J. Kim, S. Lee, and S. H. Noh, "DOGI: Data placement with oracle-guided insights for log-structured systems," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2026, pp. 543–559.

[37] X. Li, D. G. Andersen, M. Kaminsky, and M. J. Freedman, "Algorithmic improvements for fast concurrent cuckoo hashing," in *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2014.